

ReactJS 處理事件

我們將學習如何在 React 應用程式中處理事件。

處理事件基礎知識

使用 React 元素處理事件與處理 DOM 元素上的事件非常相似。

React 事件使用 camelCase 命名，而不是小寫

— 例如：寫法是 onClick 而不是 onclick。

我們將 React 事件處理程式寫在大括弧內。

在 React (/JSX) 的情況下，我們傳遞一個函數作為事件處理程式，而不是傳遞字串。

onClick={buttonClicked} 而不是 onclick="buttonClicked()"。

例如，HTML：

```
<button onclick="buttonClicked()">
```

Click the Button!

```
</button>
```

在 React 中略有不同：

```
<button onClick={buttonClicked}>
```

Click the Button!

```
</button>
```

另一個區別是，為了防止 React 的預設行為，我們不能返回 false。我們必須顯式執行 preventDefault。例如，HTML：

```
// HTML
```

```
<button onclick="console.log('Button clicked.');" return false">
```

Click Me

```
</button>
```

```
// React
```

```
function handleClick(e) {
```

```
    e.preventDefault();
```

```
    console.log('Button clicked.');
```

```
}
```

```
return (
```

```
    <button onClick={handleClick}>
```

Click me

```
</a>
```

```
);
```

在 function 元件範例中處理 onClick 事件

讓我們建立 FunctionClick 功能元件並處理 onClick 事件：

```
import React from 'react'

function FunctionClick() {
  function clickHandler() {
    console.log('Button clicked')
  }
  return (
    <div>
      <button onClick={clickHandler}>Click</button>
    </div>
  )
}
export default FunctionClick;
```

在類別元件範例中處理 onClick 事件

讓我們創建 ClassClick 類元件並處理 onClick 事件：

```
import React, { Component } from 'react'
class ClassClick extends Component {

  clickHandler() {
    console.log('Clicked the button')
  }
  render() {
    return (
      <div>
        <button onClick={this.clickHandler}>Click Me</button>
      </div>
    )
  }
}
export default ClassClick
```

綁定事件

對於 React 中的方法，`this` 關鍵字應該表示擁有該方法的元件。

讓我們使用以下建立 `EventBind` 類別元件：

```
import React, { Component } from 'react'
class EventBind extends Component {
  constructor() {
    super()
    this.state = {
      message: 'Hello'
    }
  }
  clickHandler() {
    console.log(this)
    this.setState({message: 'Goodbye'})
  }

  render() {
    return (
      <div>
        <div>{this.state.message}</div>
        <button onClick={this.clickHandler}>Click</button>
      </div>
    )
  }
}
export default EventBind
```

當您呈現此元件時，會在瀏覽器中顯示以下錯誤：

TypeError: Cannot read property 'setState' of undefined

如果沒有綁定，`this` 關鍵字將返回 `undefined`。

通常如果執行後不帶 `()` 的方法，例如 `onClick={this.handleClick}`，則應綁定該方法。我們有三項綁定事件處理程式。

1. 動態綁定：

在這裡，我們將在 `render()` 函數中執行用 `.bind(this)`。

```
import React, { Component } from 'react'

class EventBind extends Component {
  constructor() {
    super()
    this.state = {
      message: 'Hello'
    }
  }

  clickHandler() {
    console.log(this)
    this.setState({message: 'Goodbye'})
  }

  render() {
    return (
      <div>
        <div>{this.state.message}</div>
        <button onClick={this.clickHandler.bind(this)}>Click</button>
      </div>
    )
  }
}

export default EventBind
```

這種方法將起作用，但是，這種方法的問題是 `this.state.message` 中的任何更改都將導致再次重新渲染元件。

這反過來再次調用 `this.buttonClicked.bind(this)` 來綁定處理程式。

結果，將生成一個新的處理程式，該處理程式將與第一次調用 `render()` 時使用的處理程式完全不同。

2. 使用箭頭功能

這種方法比第一種方法更好：

```
import React, { Component } from 'react'
class EventBind extends Component {
  constructor() {
    super()
    this.state = {
      message: 'Hello'
    }
  }

  clickHandler = () => {
    this.setState({message:'Goodbye'})
  }

  render() {
    return (
      <div>
        <div>{this.state.message}</div>
        <button onClick={this.clickHandler}>Click</button>
      </div>
    )
  }
}

export default EventBind
```

3. 建構函數繫結：

這種方法在 **React** 應用程式中被廣泛使用，你也可以在 **React** 文件中找到這種方法：

```
import React, { Component } from 'react'
class EventBind extends Component {

  constructor() {
    super()
    this.state = {
      message: 'Hello'
    }
  }
}
```

```

    }
    this.clickHandler = this.clickHandler.bind(this)
  }

  clickHandler() {
    console.log(this)
    this.setState({message: 'Goodbye'})
  }

  render() {
    return (
      <div>
        <div>{this.state.message}</div>
        <button onClick={this.clickHandler}>Click</button>
      </div>
    )
  }
}
export default EventBind

```

第二種和第三種方法提供了良好的性能，因此廣泛用於 **React** 應用程式中以綁定事件。

傳遞參數

讓我們討論如何將參數傳遞給 **React** 中的事件處理程式。

例：使用箭頭函數將“再見”作為參數發送到 `clickHandler` 函數：

```

import React, { Component } from 'react'
class EventBind extends Component {
  constructor() {
    super()
    this.state = {
      message: 'Hello'
    }
  }

  clickHandler = (param) => {
    this.setState({message:param})
  }
}

```

```
}

render() {
  return (
    <div>
      <div>{this.state.message}</div>
      <button onClick={() => this.clickHandler("Goodbye")}>Click</button>
    </div>
  )
}
}

export default EventBind
```